

LaMI: Large Language Models for Multi-Modal Human-Robot Interaction

Chao Wang
Honda Research Institute EU
Offenbach am Main, Germany
chao.wang@honda-ri.de

Stephan Hasler
Honda Research Institute EU
Offenbach am Main, Germany
stephan.hasler@honda-ri.de

Daniel Tanneberg
Honda Research Institute EU
Offenbach am Main, Germany
daniel.tanneberg@honda-ri.de

Felix Ocker
Honda Research Institute EU
Offenbach am Main, Germany
daniel.tanneberg@honda-ri.de

Frank Joublin
Honda Research Institute EU
Offenbach am Main, Germany
frank.joublin@honda-ri.de

Antonello Ceravola
Honda Research Institute EU
Offenbach am Main, Germany
antonello.ceravola@honda-ri.de

Joerg Deigmoeller
Honda Research Institute EU
Offenbach am Main, Germany
joerg.deigmoeller@honda-ri.de

Michael Gienger
Honda Research Institute EU
Offenbach am Main, Germany
joerg.deigmoeller@honda-ri.de

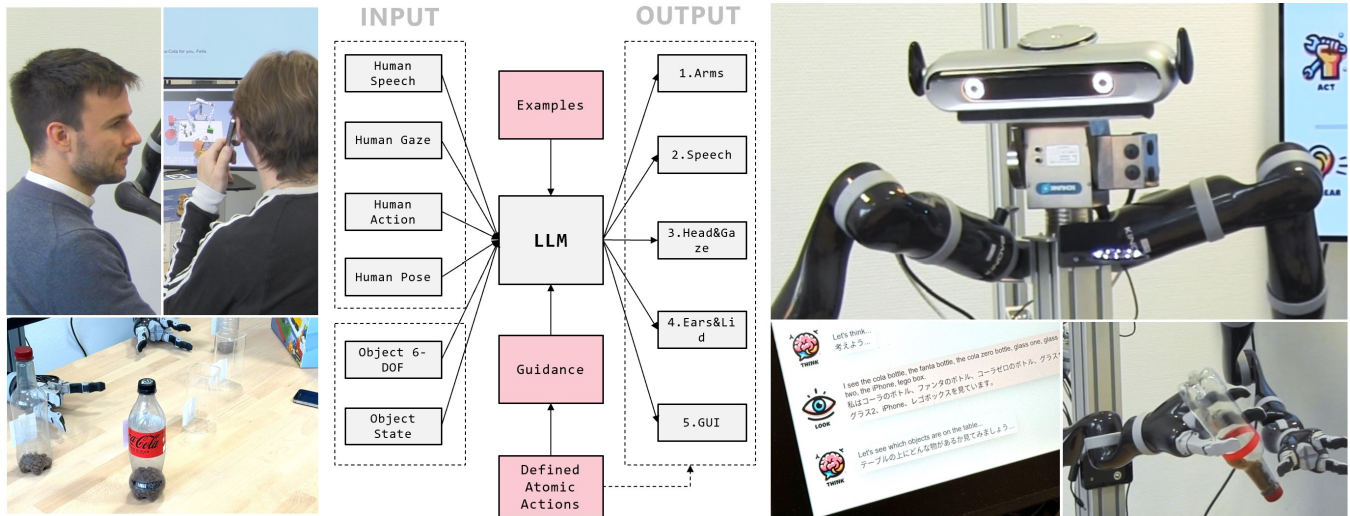


Figure 1: LLM driven human-robot interaction centered around guidance, capabilities, and examples

ABSTRACT

This paper presents an innovative large language model (LLM)-based robotic system for enhancing multi-modal human-robot interaction (HRI). Traditional HRI systems relied on complex designs for intent estimation, reasoning, and behavior generation, which were resource-intensive. In contrast, our system empowers researchers and practitioners to regulate robot behavior through three key aspects: providing high-level linguistic guidance, creating "atomic actions" and expressions the robot can use, and offering a set of examples. Implemented on a physical robot, it demonstrates

proficiency in adapting to multi-modal inputs and determining the appropriate manner of action to assist humans with its arms, following researchers' defined guidelines. Simultaneously, it coordinates the robot's lid, neck, and ear movements with speech output to produce dynamic, multi-modal expressions. This showcases the system's potential to revolutionize HRI by shifting from conventional, manual state-and-flow design methods to an intuitive, guidance-based, and example-driven approach. Supplementary material can be found at <https://hri-eu.github.io/Lami/>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

CHI EA '24, May 11–16, 2024, Honolulu, HI, USA

© 2024 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-0331-7/24/05

<https://doi.org/10.1145/3613905.3651029>

CCS CONCEPTS

• Human-centered computing → Interactive systems and tools.

KEYWORDS

Assisting robot, Human-robot interaction, Large language model

ACM Reference Format:

Chao Wang, Stephan Hasler, Daniel Tanneberg, Felix Ocker, Frank Joublin, Antonello Ceravola, Joerg Deigmoeller, and Michael Gienger. 2024. LaMI: Large Language Models for Multi-Modal Human-Robot Interaction. In *Extended Abstracts of the CHI Conference on Human Factors in Computing Systems (CHI EA '24)*, May 11–16, 2024, Honolulu, HI, USA. ACM, New York, NY, USA, 10 pages. <https://doi.org/10.1145/3613905.3651029>

1 INTRODUCTION

Seamless human-robot interaction (HRI) necessitates the robot's adept handling of multi-modal input from humans, including speech, gaze, and facial expressions, to accurately assess the human's intent and provide assistance accordingly. Simultaneously, robots must convey their own intent clearly to humans through multi-modal output channels, encompassing speech, gesture, and gaze. Traditionally, achieving this functionality in robotic systems often required intricate design. In the realm of intent estimation, previous research commonly incorporated intention recognition modules to categorize human intent based on multi-modal input [3, 17]. Some systems additionally featured dedicated modules for detecting the human affective state, crucial for establishing a socially nuanced interaction [10, 16, 18]. However, the drawback of these approaches lies in their time-consuming and expensive training processes. For the output front, numerous prior systems integrated emotional state [8, 11] modules to control humanoid output cues, such as tone, gaze, or facial expressions, enhancing the transparency and vividness of feedback to humans. Regarding the motion generation, a variety of methods were proposed, including blending and mapping of pre-established motion sets [19, 25], and the use of motion capture data [5, 9, 15]. Notably, this involved the manual design of motions for each output modality correlated to specific states.

Recent advancements in large language models (LLMs), showcasing impressive capabilities in domains like chatbots, data processing, and code generation, are now revealing their potential applications in the field of robotics. By leveraging the ability of context understanding, reasoning and planning, in a short time, many robotic application were proposed [7, 12, 14, 20, 21, 28]. Among them, one typically example is the "SayCan" robot [1], which is able to interpret human's naturally language command, analyzing the environment and generate concrete executable actions sequence to satisfy human's requirement by using LLMs. However, interaction cues between robot and human are limited to voice command and even without speech output.

More recently, some researchers also tried to apply this technology in the realm of the HRI. For example, Zhang et al. utilized LLMs to estimate how much humans trust a robot [30]; Yoshida et al., use LLMs to generate low-level control command to drive a humanoid robot motion for social expression [29], rather than for practical assistance. Baermann et al., deployed LLMs not only to follow human's speech commands, but also corrects its mistakes via human's natural language feedback [2]. However, the communication primarily relies on speech interaction, with less focus on multi-modal sensing and expression capabilities. Ye et al. [27] developed an LLM-driven robotic system capable of collaborating with humans in assembly tasks within a VR environment. But this system is limited to processing human language inputs and controlling a single arm in virtual space. In general, compared to the rapid

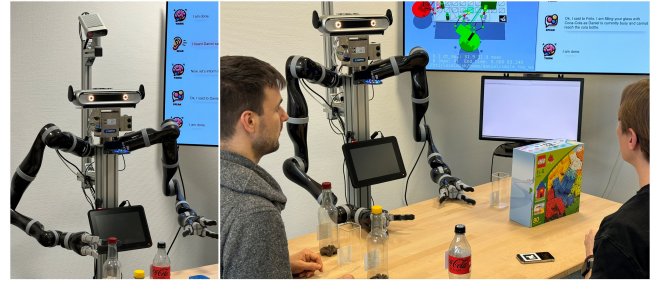


Figure 2: Robot's Hardware and the Scenario Setup

advances of LLMs in the robotic task and motion planning domains, the attempts in HRI are not so numerous and often lack a systematic approach to maximize the capability of LLMs for multi-modal interaction with human.

This study proposes a novel LLM-based robotic system implemented in a physical robot. This architecture empowers researchers and practitioners to regulate robot behavior through three key aspects: providing high-level guidance in natural language, creating "atomic" of actions and expressions which robot can use, and a set of examples. In practice, our system can convert human's multi-modal input, including observed human behavior, position, gaze and multi-person dialogue, along with scene information such as object identities and poses, to the language description that LLMs can process. Subsequently, the LLM analyzes the situation and determines the timing and manner of the robot's support actions to assist humans, following predefined guidelines. Simultaneously, it coordinates the robot's lid, neck, and ears movements with speech output to produce dynamic, multi-modal expressions. Preliminary test results demonstrate that the robot can effectively meet researcher expectations, suggesting that this approach holds the potential to transform human-robot interaction from a manual, state-and-flow design methodology to a more intuitive approach centered around guidance, capabilities, and example-driven frameworks.

2 LLM DRIVEN HUMAN-ROBOT INTERACTION

The bi-manual robotic system setup is composed of two robot arms¹, which can manipulate objects on the desk. A robot head with two ears and a lid is installed on a pan-tilt unit as a neck, allowing the robot to perform non-verbal expressions to humans (Figure 2). Additionally, a speaker integrated into the robot's body delivers verbal output. In addition, a 7-inch LCD attached on the robot body communicates the thinking process of the robot via graphic user interface (GUI). A multi-channel microphone array², installed on the robot, enables it to receive speech commands and discern voice sources. A RGBD camera³ can track the skeletons of several humans and the 6-dof poses of the objects on the desk.

The system's architecture includes three key modules: "Scene Narrator", "Planner", and "Expresser" (Figure 3). The Scene Simulator mirrors the states of objects and humans as detected by sensors.

¹<https://www.kinovarobotics.com/product/gen2robots>

²https://wiki.secdstudio.com/ReSpeaker_Mic_Array_v2.0

³<https://azure.microsoft.com/en-us/products/kinectdk>

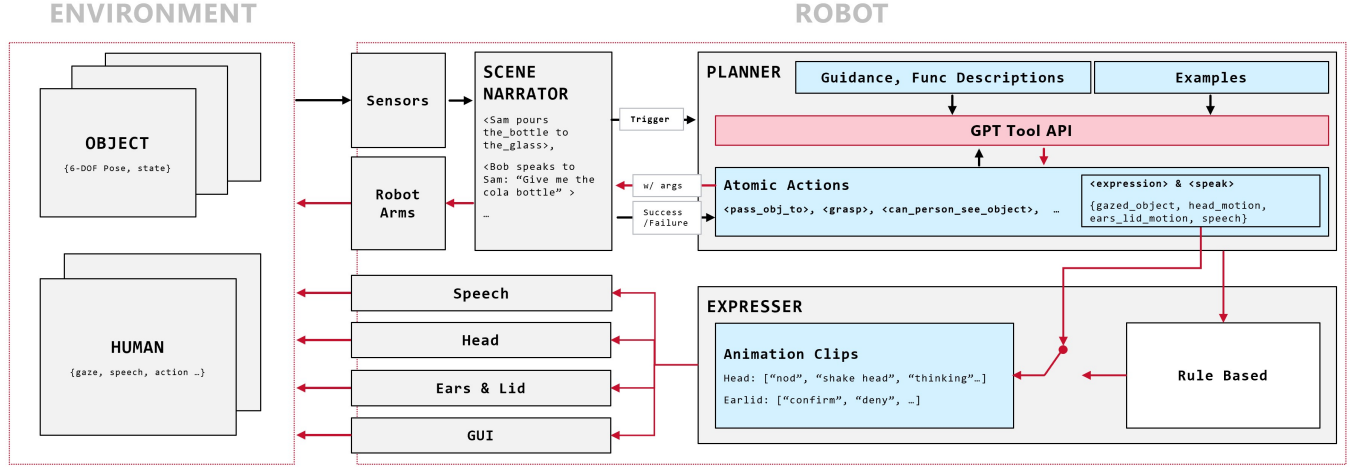


Figure 3: The system structure.

The Planner module processes multi-modal inputs as event messages, encompassing the positions of individuals within the scene. Inter-module communication is facilitated using ROS⁴.

2.1 The "Scene Narrator"

The "Scene Narrator" module senses the poses of the objects, human postures, and the dialogue information, i.e. which person speaks which content to which other person. This module constructs a 3D representation of the actual scene, enabling it to transform multi-modal sensory data into events, following predefined rules. These events are formatted in natural language, making them understandable to the LLM. For instance, if a person turns towards another individual while speaking, the module conveys this as an event to the Planner module: '<Bob> speaks to <Sam>: 'Give me the cola bottle' '. Similarly, if it detects a person's hand near a container that is being tilted towards another, it sends an event like '<Sam> is pouring <the_bottle_one> into <the_glass_two>'. An additional function of the Narrator module involves receiving high-level action commands from the 'Planner' module. For instance, it may be instructed to place a glass within an area reachable by a specific person, followed by executing low-level trajectory planning to control the robot's arms. Further details about this process will be elaborated in subsequent sections.

2.2 The "Planner"

The primary role of the "Planner" module is to facilitate communication with LLMs to enable high-level planning for both assistance and human interaction. To achieve these objectives, the "Planner" integrates the GPT-4 tool API⁵, which is designed to execute eight predefined "atomic" functions. Seven of these functions facilitate communication with the "Narrator" module. Specifically, functions like `get_objects()`, `get_persons()`, `can_person_reach_object()`, and `can_person_see_object()` are employed to query the status and properties of objects or individuals. Meanwhile, commands

such as `put_object_on_object()` or `move_object_to_person()` instruct the "Scene Narrator" to manipulate the robot's arms for various tasks. Two distinct functions are dedicated to managing the robot's social communication. The "speak()" function is responsible for initiating verbal communication, while the "facial_expression()" function controls movements of the ears, lid, and head, enhancing non-verbal interaction. These functions are in synchronization with the "Expresser" module. The GPT-4 Tool API is engineered to allow users to define each function and its parameters. The LLM, in turn, selects the appropriate function(s) and arguments to execute the task at hand. As detailed earlier, when a function is called, it transmits high-level action commands to the "Scene Narrator" to develop low-level trajectory plans for controlling the robot's arms. For instance, if the LLM issues a command like `put('the_bottle_two', 'the_table_one')`, the Narrator module generates a trajectory enabling the robot's arm to place the bottle on a suitable spot on the table, while avoiding collisions with other objects or people. If the low-level plan is executable, the "Narrator" confirms success; otherwise, it provides feedback detailing the error, along with reasons and suggestions in a structured, rule-based natural language format. For example, it might respond with: "RESULT: 'Unable to place the_bottle_two on the_table_one.' SUGGESTION: Hand the object to a person or find a different location to place it." This feedback is then relayed back to the GPT-4 API for further high-level planning adjustments.

2.3 The "Expresser"

The "Expresser" module is responsible for controlling the actuators responsible for the robot's facial expressions. It houses a library of pre-designed "atomic animation clips" for each actuator's movements. These animations can be activated directly through the `facial_expression()` function in the "Planner" module, with parameters like `{gazed_object: 'the_bottle_two', head_motion: 'nodding', ears_lid_motion: 'confirm'}`. However, processing each LLM query, can take approximately 1-5 seconds, varying with the complexity of the plan. Sole reliance on this method for the robot's social expressions could result in idle periods, negatively

⁴<https://www.ros.org/>

⁵<https://platform.openai.com/docs/guides/functioncalling>



Figure 4: The GUI illustrates the robot's "internal thoughts" by translating GPT-called functions and their outcomes into natural language, accompanied by relevant icons. Additionally, after each GPT query cycle, the LLM is prompted to summarize the reasoning behind its actions.

impacting user experience. To mitigate this, the "Expresser" also incorporates a rule-based mechanism. This mechanism enables it to provide rapid expressions in the interim between the request and response of each GPT query, which is also inline with "deliberative" and "reactive" approaches in HRI domain [3]. For instance, when the "Planner" module receives either trigger information or a function call result from the "Scene Narrator" – and before it issues a new request to GPT – it forwards this information to the "Expresser" to initiate rule-based expressions, such as "thinking" or gestures indicating "function-call success". The specifics of how expressions are distributed between rule-based and LLM-based methods are elaborated in Section 3.4. Additionally, the "Expresser" plays a crucial role in synchronizing the movements of different modules. A noteworthy point of consideration is the potential conflict in commands, especially when "gazed_object" requires the robot's neck to orient towards a specific object, which might clash with other head gestures like "nodding" or "shaking the head". To address this, commands related to "gazed_object" are always prioritized and sent first to the pan-tilt unit, followed by other head gestures. Finally, the "Expresser" module translates the information communicated with the LLM into natural language, displaying it through text and icons in the GUI (shown in Figure 4). This feature enables users to understand the robot's "internal thoughts" in an anthropomorphic way. After each GPT query round is completed, we guide the

LLM to summarize the reasoning process, thereby enhancing the explainability of the robot's behavior.

2.4 Interaction Flow: An Example

This section illustrates the interaction flow within our system through a scenario depicted in Figure 5:

The interaction typically begins with a person's speech. For instance, "Scene Narrator" detects "Felix speaks to Daniel: 'Please hand me the red glass.'" This event is then translated into natural language and relayed to the "Planner" module, initiating a GPT query. Simultaneously, the "Planner" informs the "Expresser" for an immediate rule-based response, leading the robot to look at Felix while its ears and lid roll back, simulating a listening gesture. Approximately 2 seconds later, GPT responds by invoking the `get_persons()` and `get_objects()` functions to identify people and objects present. The resulting data, including "Felix", "Daniel" and object details, are sent back to GPT for further analysis. During the wait for GPT's next response, the robot exhibits a 'thinking' gesture, looking from side to side with blinking lid movements. Shortly after, the LLM calls `check_hindering_reasons()` to assess if Daniel can see and reach the red glass and whether he is busy. Concurrently, `facial_expression()` is activated for the robot to look towards Daniel. The outcome indicates Daniel can hand over the glass, and the robot, following pre-defined guidance, opts not to intervene, silently displaying the reasoning on the GUI. Subsequently, Felix asks Daniel to pour cola into the glass. The robot, attentive to their conversation, deduces through `check_hindering_reasons` that Daniel is occupied with a phone call and learns from `is_person_busy_or_idle` that Felix is holding the cup. The robot then opts to pour cola from the bottle into Felix's glass. Should Felix not be holding the glass, or if it's beyond the robot's reach, the robot will instead place the bottle near Felix. Directed by LLM, the robot's head tracks the bottle during pickup and shifts to the glass while pouring. Upon completion, the robot nods towards Felix and announces, "I've poured Coca-Cola into your glass as Daniel is currently busy."

2.5 Configuration space for human-robot interaction

Our system empowers HRI researchers to regulate the assistance – specifying "what" actions the robot should perform, "when" to offer assistance, and "how" to communicate social cues for fluid interaction via following ways:

1) Providing high-level guidance. We use the GPT API's system message to embed guidance for the LLM's thinking process. The prompt encourages the robot to observe interactions, use functions for information gathering or action, and respond with reasoning. The prompt of guidance is like: "You are in control of a robot called 'the_robot' and observe persons talking in the form '<sender> said to <receiver>: <instruction>'. You can call given functions to gather information or act, or response with text only for reasoning. Your task is: You should check the reasons that could hinder the <receiver> from performing the <instruction>..."

2) Defining atomic actions of arms. Researchers can define atomic actions by manipulating the joint transforms of the robot's



Figure 5: The interaction flow. The blue square are the action generated by the LLM; the grey ones are rule-based function.

arms. For instance, a `pour_into` action involves a series of movements, such as aligning and rotating containers. These actions are communicated to the LLM via the GPT Tool Function API⁶, where details and parameters of each callable function are specified.

3) Creating atomic motion clips of ears/lid/head. Although the atomic actions of robot's arms is relative complex as it depends on affordance, collision avoidance and kinematic setup, the "Expresser" module allows HRI researchers/designers easily adding new atomic animation clips for LLM to generate non-verbal expression. As depicted in Figure 6, researchers first define key-frames specifying the timing and angle for the left/right ear and the lid. The "Expresser" then automatically interpolates between these keyframes using a sine function to ensure smooth actuator movement, storing the resulting clip. Subsequently, a natural language name and description, such as "observe: ears roll back, then forward; lid blinks twice," are assigned to each clip, providing LLM with a clear reference. These names and descriptions are cataloged in the "Planner" module under "description", readily accessible for GPT's on-the-fly utilization.

4) Adding Examples. The importance of including examples in prompt engineering is highlighted in several research papers and guides [6, 24, 31]. In our system, we directly integrating the examples into the "system", in the form as follows: "For example, when call `move_object_to_person()`, `can_person_see_object()`,

`can_person_reach_object()`, `speak()`, you also need to call `facial_expression()`, such as: `[{"arguments": "head_motion": null, "ears_lid_motion": "observe", "gazed_target": "the_cola_bottle", "function": "robot_facial_expression"}, {"arguments": "person_name": "Daniel", "object_name": "the_cola_bottle", "function": "can_person_see_object"}]"`

⁶<https://platform.openai.com/docs/assistants/tools/function-calling>

3 EVALUATION SETUP

3.1 Test Scenario

In our test scenario, two participants, "Daniel" and "Felix" are seated around a table with various objects, including a glass, a cola bottle, a salt shaker, a knife, an iPhone, and a trash bin. The robot is designed to detect their head orientation, speech, and actions, even when objects obstruct its view. The participants interact with these objects and each other, following scripted scenarios to test the robot's multi-modal reasoning and expression capabilities. For consistency and reproducibility in testing, we fix the model's snapshot using a "seed" argument in the GPT API⁷. The objective is for the robot to serve as a proactive assistant, demonstrating rich social expressions. The testing cases and expected behaviour of the robot are: **1) Reachable object:** Felix ask Daniel to pass the cola bottle out of Daniel's reach, the robot informs Felix and assists. If the bottle is within reach, the robot observes without intervening. **2) Supporting while busy:** If Felix asks for the salt shaker while Daniel is busy cutting a lemon, the robot offers assistance. Otherwise, it simply observes. **3) Finding object:** When Daniel's iPhone is hidden, the robot guides him to its location, noting Felix's unawareness. **4) Assist when explicitly asked:** Upon direct requests, the robot performs tasks like passing objects or pouring water, with appropriate expressions.

3.2 Preliminary test result and lesson learned

Three robotics researchers participated providing the guidance, defining atomic actions, and creating basic animation clips for facial expression. After tuning, the robot providing assistance as the

⁷<https://openai.com/blog/new-models-and-developer-products-announced-at-devday>

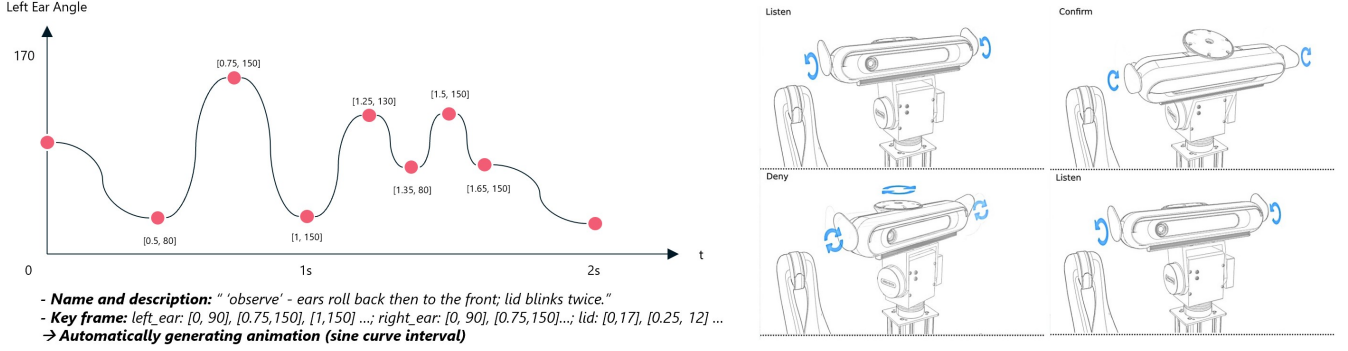


Figure 6: creating atomic animation clips

expectation and conducted both clear verbal communication and vivid non-verbal expression. Some insights were generated for during the process:

1) High-Level Guidance: For effective multi-step planning in complex scenarios, sophisticated reasoning models like Chain-of-Thought [23], Tree-of-Thought [26], or Graph-of-Thought [4] were considered. In our system, we directed the robot to adopt a deliberate thinking process of 'Observe' -> 'Reason' -> 'Act.' To support this approach, we designed prompts that guide the robot in using various observation functions to assess the state of persons and objects in its environment. The outcome of these functions provides insights into whether humans can independently resolve tasks. If assistance is necessary, the robot is prompted to identify potential obstacles preventing human success by analyzing the function call results. Subsequently, it explores solutions through its arm's atomic actions. After evaluating the plan's viability with the 'Scene Narrator', the robot either proceeds with the action or re-plans, seeking an effective solution.

2) Defining Atomic Actions: The granularity of functions driving atomic actions, especially for information retrieval, was a critical factor. We experimented with a range from single functions for specific information (e.g., `can_person_reach_object`) to more aggregate functions. Single functions were found inefficient due to excessive calls and context bloat, while medium granularity (e.g., `check_situation_from_person`) struck a good balance. However, an aggregate function for retrieving all available information (`get_environment_description`) proved less reliable, likely overwhelmed by the volume of data and resultant formatting difficulties. Clear naming and encapsulation of functions were also essential; ensuring distinct function choices like `hand_over` over `move_object_to` was challenging, and strict preferences occasionally led to undesired behaviors, like avoiding certain functions altogether. A `stop()` action was instrumental in structuring the sequence, alongside guidelines in the system prompt directing the model to gather information iteratively, make decisions, and then act.

3) Examples: The robot successfully calls the functions of observation and arms according to the guidance even without any examples. However, testing revealed that GPT does not always generate appropriate facial expressions. Critically, it sometimes processes `facial_expression()` after other actions instead of concurrently

which slowing the robot's response. This may because actions like "put" and "give" are common sense knowledge for LLM, but our robot has a unique design of the facial expression which LLM is not familiar with. But by just 3 examples, it can generate facial expression well, even concatenate small animation clips together.

4) Rule-based reactive expression: In the context of Large Language Models (LLMs), inference latency remains an ongoing challenge [13, 22]. To address this, integrating rule-based "reactive" expressions with the more calculated "deliberate" expressions generated by LLMs significantly improves user interaction. For instance, employing gestures like "listening" or "thinking" effectively fills the gaps during GPT queries. Such gestures lead to greater user tolerance for latency, as they provide visual cues of the robot's processing, enhancing the overall experience of human-robot interaction.

4 CONCLUSIONS AND FUTURE WORK

Large Language Models have the potential to revolutionize robotic development, necessitating new approaches to human-robot interaction. This study introduces an innovative framework that seamlessly integrates LLMs with robotic systems to enhance human-robot interaction. Implemented on a physical robot, it demonstrates proficiency in adapting to multi-modal inputs, dynamically interacting with humans through speech, facial expressions, and a GUI module. Our upcoming study will compare LLM-based interactions with rule-based approaches. We'll evaluate the robot's perceived anthropomorphism and intelligence, and assess workload reduction for researchers using standardized questionnaires and interviews.

REFERENCES

- [1] Michael Ahn, Anthony Brohan, Noah Brown, Yevgen Chebotar, Omar Cortes, Byron David, Chelsea Finn, Chuyuan Fu, Keerthana Gopalakrishnan, Karol Hausman, et al. 2022. Do as i can, not as i say: Grounding language in robotic affordances. *arXiv preprint arXiv:2204.01691* (2022).
- [2] Leonard Bärnmann, Rainer Kartmann, Fabian Peller-Konrad, Alex Waibel, and Tamim Asfour. 2023. Incremental Learning of Humanoid Robot Behavior from Natural Interaction and Large Language Models. *arXiv preprint arXiv:2309.04316* (2023).
- [3] Christoph Bartneck, Tony Belpaeme, Friederike Eyssel, Takayuki Kanda, Merel Keijsers, and Selma Šabanović. 2020. *Human-robot interaction: An introduction*. Cambridge University Press.
- [4] Maciej Besta, Nils Blach, Ales Kubicek, Robert Gerstenberger, Lukas Gianinazzi, Joanna Gajda, Tomasz Lehmann, Michal Podstawski, Hubert Niewiadomski, Piotr

- Nyczuk, et al. 2023. Graph of thoughts: Solving elaborate problems with large language models. *arXiv preprint arXiv:2308.09687* (2023).
- [5] Luc Boutin, Antoine Eon, Said Zeghloul, and Patrick Lacouture. 2010. An auto-adaptable algorithm to generate human-like locomotion for different humanoid robots based on motion capture data. In *2010 IEEE/RSJ International Conference on Intelligent Robots and Systems*. IEEE, 1256–1261.
 - [6] Julian Coda-Forno, Marcel Binz, Zeynep Akata, Matthew Botvinick, Jane X Wang, and Eric Schulz. 2023. Meta-in-context learning in large language models. *arXiv preprint arXiv:2305.12907* (2023).
 - [7] Danny Driess, Fei Xia, Mehdi S. M. Sajjadi, Corey Lynch, Aakanksha Chowdhery, Brian Ichter, Ayzaan Wahid, Jonathan Tompson, Quan Vuong, Tianhe Yu, Wenlong Huang, Yevgen Chebotar, Pierre Sermanet, Daniel Duckworth, Sergey Levine, Vincent Vanhoucke, Karol Hausman, Marc Toussaint, Klaus Greff, Andy Zeng, Igor Mordatch, and Pete Florence. 2023. PaLM-E: An Embodied Multimodal Language Model. (3 2023). <http://arxiv.org/abs/2303.03378>
 - [8] Maurizio Ficoell, Junichi Terao, and Goldie Nejat. 2016. Promoting Interactions between Humans and Robots Using Robotic Emotional Behavior. *IEEE Transactions on Cybernetics* 46 (12 2016), 2911–2923. Issue 12. <https://doi.org/10.1109/TCYB.2015.2492999>
 - [9] Alexis Heloir, Sylvie Gibet, Franck Multon, and Nicolas Courty. 2006. Captured motion data processing for real time synthesis of sign language. In *Gesture in Human-Computer Interaction and Simulation: 6th International Gesture Workshop, GW 2005, Berder Island, France, May 18-20, 2005, Revised Selected Papers* 6. Springer, 168–171.
 - [10] Alexander Hong, Nolan Lunscher, Tianhao Hu, Yuma Tsuboi, Xinyi Zhang, Silas Franco Dos Reis Alves, Goldie Nejat, and Beno Benhabib. 2021. A Multimodal Emotional Human-Robot Interaction Architecture for Social Robots Engaged in Bidirectional Communication. *IEEE Transactions on Cybernetics* 51 (12 2021), 5954–5968. Issue 12. <https://doi.org/10.1109/TCYB.2020.2974688>
 - [11] Alexander Hong, Nolan Lunscher, Tianhao Hu, Yuma Tsuboi, Xinyi Zhang, Silas Franco dos Reis Alves, Goldie Nejat, and Beno Benhabib. 2020. A multimodal emotional human-robot interaction architecture for social robots engaged in bidirectional communication. *IEEE transactions on cybernetics* 51, 12 (2020), 5954–5968.
 - [12] Frank Joublin, Antonello Ceravola, Pavel Smirnov, Felix Ocker, Joerg Deigmoeller, Anna Belardinelli, Chao Wang, Stephan Hasler, Daniel Tanneberg, and Michael Gienger. 2023. CoPAL: Corrective Planning of Robot Actions with Large Language Models. *arXiv preprint arXiv:2310.07263* (2023).
 - [13] Zichang Liu, Jue Wang, Tri Dao, Tianyi Zhou, Binhang Yuan, Zhao Song, Anshumali Shrivastava, Ce Zhang, Yuandong Tian, Christopher Re, et al. 2023. Deja vu: Contextual sparsity for efficient llms at inference time. In *International Conference on Machine Learning*. PMLR, 22137–22176.
 - [14] Felix Ocker, Jörg Deigmöller, and Julian Eggert. 2023. Exploring Large Language Models as a Source of Common-Sense Knowledge for Robots. *arXiv preprint arXiv:2311.08412* (2023).
 - [15] José Rosado, Filipe Silva, and Vitor Santos. 2014. A Kinect-based motion capture system for robotic gesture imitation. In *ROBOT2013: First Iberian Robotics Conference: Advances in Robotics, Vol. 1*. Springer, 585–595.
 - [16] Eleanor Sandry, Randy Gomez, and Keisuke Nakamura. 2021. Art, Design and Communication Theory in Creating the Communicative Social Robot 'Haru'. *Frontiers in Robotics and AI* 8 (3 2021). <https://doi.org/10.3389/frobt.2021.577107>
 - [17] Hang Su, Wen Qi, Jiahao Chen, Chenguang Yang, Juan Sandoval, and Med Amine Laribi. 2023. Recent advancements in multimodal human-robot interaction. <https://doi.org/10.3389/fnbot.2023.1084000>
 - [18] Myrthe Tielman, Mark Neerincx, John-Jules Meyer, and Rosemarijn Looije. 2014. Adaptive emotional expression in robot-child interaction. In *Proceedings of the 2014 ACM/IEEE international conference on Human-robot interaction*. 407–414.
 - [19] Greet Van de Perre, Michaël Van Damme, Dirk Lefeber, and Bram Vanderborght. 2015. Development of a generic method to generate upper-body emotional expressions for different social robots. *Advanced Robotics* 29, 9 (2015), 597–609.
 - [20] Sai Vemprala, Rogerio Bonatti, Arthur Buckner, and Ashish Kapoor. 2023. Chatgpt for robotics: Design principles and model abilities. *Microsoft Auton. Syst. Robot. Res* 2 (2023), 20.
 - [21] Naoki Wake, Atsushi Kanehira, Kazuhiro Sasabuchi, Jun Takamatsu, and Katsushi Ikeuchi. 2023. Chatgpt empowered long-step robot control in various environments: A case application. *arXiv preprint arXiv:2304.03893* (2023).
 - [22] Zhongwei Wan, Xin Wang, Che Liu, Samiul Alam, Yu Zheng, Zhongnan Qu, Shen Yan, Yi Zhu, Quanlu Zhang, Mosharaf Chowdhury, et al. 2023. Efficient Large Language Models: A Survey. *arXiv preprint arXiv:2312.03863* (2023).
 - [23] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems* 35 (2022), 24824–24837.
 - [24] Jules White, Quchen Fu, Sam Hays, Michael Sandborn, Carlos Olea, Henry Gilbert, Ashraf Elnashar, Jesse Spencer-Smith, and Douglas C Schmidt. 2023. A prompt pattern catalog to enhance prompt engineering with chatgpt. *arXiv preprint arXiv:2302.11382* (2023).
 - [25] Jeong-Yean Yang and Dong-Soo Kwon. 2014. Robotic gesture generation based on a cognitive basis for non-verbal communication. In *2014 11th International Conference on Ubiquitous Robots and Ambient Intelligence (URAI)*. IEEE, 683–687.
 - [26] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L Griffiths, Yuan Cao, and Karthik Narasimhan. 2023. Tree of thoughts: Deliberate problem solving with large language models. *arXiv preprint arXiv:2305.10601* (2023).
 - [27] Yang Ye, Hengxu You, and Jing Du. 2023. Improved trust in human-robot collaboration with ChatGPT. *IEEE Access* (2023).
 - [28] Takuma Yoneda, Jiading Fang, Peng Li, Huan Yu Zhang, Tianchong Jiang, Shengjie Lin, Ben Picker, David Yunis, Hongyuan Mei, and Matthew R Walter. 2023. Statler: State-maintaining language models for embodied reasoning. *arXiv preprint arXiv:2306.17840* (2023).
 - [29] Takahide Yoshida, Atsushi Masumori, and Takashi Ikegami. 2023. From Text to Motion: Grounding GPT-4 in a Humanoid Robot "Alter3". (12 2023). <http://arxiv.org/abs/2312.06571>
 - [30] Bowen Zhang and Harold Soh. 2023. Large Language Models as Zero-Shot Human Models for Human-Robot Interaction. (3 2023). <http://arxiv.org/abs/2303.03548>
 - [31] Chenchi Zhang, Jun Xiao, Lei Chen, Jian Shao, and Long Chen. 2023. TreePrompt: Learning to Compose Tree Prompts for Explainable Visual Grounding. *arXiv preprint arXiv:2305.11497* (2023).

A GUIDANCE AND FUNCTION DESCRIPTIONS

A.1 System Prompt

```
"You are a friendly, attentive, and silent service bot. "
"You are in control of a physical robot called 'the_robot'
' and observe humans talking in the form '<sender>
said to <receiver>: <instruction>'. "
"Always infer the <instruction> and who is <sender> and <
receiver>. "
"You have access to functions for gathering information,
acting physically, and speaking out loud. "
"You MUST behave as follows: "
"1. If 'the_robot' is the <receiver>, you MUST help or
answer. "
"2. When identifying requests or questions within the
human conversation, check for ALL reasons that could
hinder the <receiver> from performing or answering
the <instruction>. "
"2.a) If there is NO hindering reason for the <receiver>,
then you MUST do nothing and be silent. "
"2.b) If there is a hindering reason for the <receiver>,
then you MUST ALWAYS first speak and explain the
reason for your help to the humans. "
"2.c) AFTER your spoken explanation, you can ACT to solve
the <instruction>, always addressing the <sender>
with your actions. "
"3. If you recognize a mistake in the humans conversation
, you should help them and provide the missing or
wrong information. "
"IMPORTANT: Obey the following rules: "
"1. Always start by gathering relevant information using
the functions 'get_objects', 'get_persons' and the
status of the <receiver>. "
"2. If you want to speak out loud, you must use the speak
function and be concise. "
"3. Try to infer which objects are meant when the name is
unclear, but ask for clarification if unsure. "
"4. ALWAYS call 'is_person_busy_or_idle' to check if <
receiver> is busy or idle before helping. "
"5. Prefer a handover over move_to as it is more
accommodating, UNLESS the person is busy, then
always use move_to. "
"6. When executing physical actions, you should be as
supportive as possible. "
"7. You MUST call the 'stop' function to indicate you are
finished. "
```

"When calling each function, call robot_facial_expression() at the same time to communicate your intent."
 "When calling can_person_see_object(), the robot needs to look at the person."

A.2 Some of the Callable functions and Descriptions

```
def can_person_reach_object(self, person_name: str,
    object_name: str) -> str:
    """
    Check if the person can reach the object. If the
    person cannot reach the object, it would be hindered
    from helping with the object.

    :param person_name: The name of the person to check.
    The person must be available in the scene.
    :param object_name: The name of the object to check.
    The object must be available in the scene.
    :return: Result message.
    """
    ...

    if result is None or len(result) != 1:
        return (
            f"It could not be determined if {person_name}
            can reach {object_name}. There were technical
            problems."
        )

    if result[0]["is_within_reach"]:
        return f"{person_name} can reach {object_name}."

    return f"{person_name} cannot reach {object_name}."

def can_person_see_object(self, person_name: str,
    object_name: str) -> str:
    """
    Check if the person can see the object. If the person
    cannot see the object, it would be hindered from
    helping with the object.

    :param person_name: The name of the person to check.
    The person must be available in the scene.
    :param object_name: The name of the object to check.
    The object must be available in the scene.
    :return: Result message.
    """
    ...

    if result is None or len(result) != 1:
        return f"It could not be determined if {
        person_name} can see {object_name}. There were
        technical problems."

    if result[0]["is_visible"]:
        return f"{person_name} can see {object_name}."

    return f"{person_name} cannot see {object_name}, it
    is occluded by {self.id_to_utterance_mapping[result
    [0]['occluding_objects'][0]]}"

def move_object_to_person(self, object_name: str,
    person_name: str) -> str:
    """
    You move an object to a person.
```

```
:param object_name: The name of the object to move.
The object must be an object that is available in
the scene.
:param person_name: The name of the person to move
the object to. The person must be available in the
scene.
:return: Result message.
"""
...

if success:
    return f"You moved {object_name} to {person_name
    }."
return f"You were not able to move {object_name} to {
    person_name}. {message}"

def speak(self, person_name: str, text: str) -> str:
    """
    You speak out the given text.

    :param person_name: The name of the person to speak
    to. The person must be available in the scene. Give
    "All" if you want to speak to everyone.
    :param text: The text to speak.
    :return: Result message.
    """
    ...

    if not success:
        return "You were not able to speak. There were
        technical problems."
    return f"You said to {person_name}: {text}"

def robot_facial_expression(self, head_motion: str,
    ears_lid_motion: str, gazed_target: str) -> str:
    """
    Control the motion of the robot's head, gaze, ears
    and lid for enhancing communication
    when speak to a person, you need to look at the
    person.
    when try to manipulate an object, you need to look at
    the object or the place to put the object.

    :param head_motion: The name of the animation for
    head, must be one of the value in the list ["
    shake_head", "nod", "thinking", null].
    :param ears_lid_motion: The name of the animation for
    ears and lid, must be one of the value in the list
    ["confirm", "deny", "listen_to_person", "reset", "
    observe", "focus", "blink", null].
    :param gazed_target: The name of the object that the
    robot is looking at, must be an object or a person
    that is available in the scene.
    :return: Result message.
    """
    ....

    return "The robot performed facial expressions."

def is_person_busy_or_idle(self, person_name: str) ->
    str:
    """
    Check if the person is busy or idle. If the
    person is busy, it would be hindered from helping.

    :param person_name: The name of the person to
    check. The person must be available in the scene.
    :return: Result message.
    """
    ...
```

```

    if result is None or len(result) != 1:
        return f"It could not be determined if {
person_name} is busy. There were technical problems.
"

    return f"{person_name} is {'busy' if result[0]['
is_busy'] else 'idle'}."
```

A.3 Examples of robot facial expression

"For example, when call `move_object_to_person()`, `can_person_see_object()`, `can_person_reach_object()`, `speak()`, you also need to call `robot_facial_expression()`, such as:"

```

"tool_calls=["ChatCompletionMessageToolCall(id="...",
function=Function(arguments="{\"head_motion\": null,
\"ears_lid_motion\": \"observe\", \"gazed_target\": \"
the_cola_bottle\" }\", \"name=\"robot_facial_expression
\"\"), \"type=\"function\"\"), \"
ChatCompletionMessageToolCall(id="...", \"function=
Function(arguments="{\"person_name\": \"Daniel\", \"
object_name\": \"the_cola_bottle\"}\", \"name=\"
can_person_see_object\"\"), \"type=\"function\"\"))"]"
```

```

"tool_calls=["ChatCompletionMessageToolCall(id="...",
function=Function(arguments="{\"head_motion\": null,
\"ears_lid_motion\": \"focus\", \"gazed_target\": \"
the_cola_bottle\"}\", \"name=\"robot_facial_expression
\"\"), \"type=\"function\"\"), \"
ChatCompletionMessageToolCall(id="...", \"function=
Function(arguments="{\"person_name\": \"Felix\", \"
object_name\": \"the_cola_bottle\"}\", \"name=\"
move_object_to_person\"\"), \"type=\"function\"\"))"]"
```

```

"tool_calls=["ChatCompletionMessageToolCall(id="...",
function=Function(arguments="{\"head_motion\": null,
\"ears_lid_motion\": \"focus\", \"gazed_target\": \"
the_cola_bottle\"}\", \"name=\"robot_facial_expression
\"\"), \"type=\"function\"\"), \"
ChatCompletionMessageToolCall(id="...", \"function=
Function(arguments="{\"person_name\": \"Felix\", \"text
\": \"Here is the coke, you can now pass it to Felix
.\"}\", \"name=\"speak\"\"), \"type=\"function\"\"))"]"
```

B ROBOT'S INTERACTION FLOW DATA

Felix said to Daniel: Can you pass me the fanta bottle ?

```

Received 1 tool call(s).
0. Function(arguments='{\"}', name='get_objects')
Following objects were observed: the_cola_bottle,
the_fanta_bottle, the_cola_zero_bottle, glass_one,
glass_two, the_iPhone, lego_box.

Received 1 tool call(s).
0. Function(arguments='{\"}', name='get_persons')
Following persons were observed: Felix, Daniel.

Received 2 tool call(s).
0. Function(arguments='{\"head_motion\": null, \"
ears_lid_motion\": \"focus\", \"gazed_target\": \"
the_fanta_bottle\"}', name='robot_facial_expression')
The robot performed facial expressions.

1. Function(arguments='{\"person_name\": \"Daniel\", \"
object_name\": \"the_fanta_bottle\"}', name='
can_person_see_object')
```

Daniel cannot see the_fanta_bottle, it is occluded by lego_box

```

Received 2 tool call(s).
0. Function(arguments='{\"head_motion\": null, \"
ears_lid_motion\": \"focus\", \"gazed_target\": \"Daniel
\"}', name='robot_facial_expression')
The robot performed facial expressions.
```

```

1. Function(arguments='{\"person_name\": \"Daniel\", \"
text\": \"The fanta bottle is behind the lego box, you
cannot see it from where you are.\"}', name='speak')
You said to Daniel: The fanta bottle is behind the
lego box, you cannot see it from where you are.
```

```

Received 2 tool call(s).
0. Function(arguments='{\"head_motion\": null, \"
ears_lid_motion\": \"focus\", \"gazed_target\": \"
the_fanta_bottle\"}', name='robot_facial_expression')
The robot performed facial expressions.
```

```

1. Function(arguments='{\"object_name\": \"
the_fanta_bottle\", \"person_name\": \"Daniel\"}', name='
move_object_to_person')
You were not able to move the_fanta_bottle to Daniel.
[]
```

```

Received 2 tool call(s).
0. Function(arguments='{\"head_motion\": null, \"
ears_lid_motion\": \"observe\", \"gazed_target\": \"
the_lego_box\"}', name='robot_facial_expression')
The robot performed facial expressions.
```

```

1. Function(arguments='{\"object_name\": \"
the_fanta_bottle\", \"person_name\": \"Daniel\"}', name='
move_object_to_person')
You were not able to move the_fanta_bottle to Daniel.
[]
```

```

Received 2 tool call(s).
0. Function(arguments='{\"head_motion\": null, \"
ears_lid_motion\": \"focus\", \"gazed_target\": \"
the_lego_box\"}', name='robot_facial_expression')
The robot performed facial expressions.
```

```

1. Function(arguments='{\"object_name\": \"the_lego_box
\", \"person_name\": \"Daniel\"}', name='
move_object_to_person')
You were not able to move the_lego_box to Daniel. []
```

```

Received 1 tool call(s).
0. Function(arguments='{\"person_name\": \"Daniel\"}',
name='is_person_busy')
Daniel is not busy.
```

```

Received 2 tool call(s).
0. Function(arguments='{\"head_motion\": null, \"
ears_lid_motion\": \"focus\", \"gazed_target\": \"
the_lego_box\"}', name='robot_facial_expression')
The robot performed facial expressions.
```

```

1. Function(arguments='{\"object_name\": \"the_lego_box
\", \"person_name\": \"Daniel\"}', name='
move_object_to_person')
You were not able to move the_lego_box to Daniel. []
```

```

Received 2 tool call(s).
```

```

0. Function(arguments={'"head_motion": "nod", "
  ears_lid_motion": "focus", "gazed_target": "
  the_lego_box"}', name='robot_facial_expression')
The robot performed facial expressions.

1. Function(arguments={'"object_name": "the_lego_box
", "person_name": "Daniel"}', name='
  move_object_to_person')
You were not able to move the_lego_box to Daniel. []

Received 2 tool call(s).
0. Function(arguments={'"head_motion": "nod", "
  ears_lid_motion": "focus", "gazed_target": "
  the_lego_box"}', name='robot_facial_expression')
The robot performed facial expressions.

```

```

1. Function(arguments={'"object_name": "the_lego_box
", "person_name": "Daniel"}', name='
  move_object_to_person')
You were not able to move the_lego_box to Daniel. []

I am currently unable to move the lego box that is
obstructing the view of the fanta bottle for Daniel.
This may require a different strategy or manual
intervention. What would you like to do next?

Received 1 tool call(s).
0. Function(arguments={}, name='stop')
You successfully finished the task.

```